

Playwright Feature Spotlight: Parallel Execution (Advanced Guide)

Problem in Large Automation Projects:

- Test suites take too long to execute.
- CI/CD pipelines become slow.
- Delayed feedback to developers.
- Infrastructure cost increases.

Solution: Built-in Parallel Execution

- Runs tests using multiple workers.
- Each test runs in isolated browser context.
- No additional libraries required.
- Significantly reduces execution time.

Example 1: Configure Workers in playwright.config.js

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  workers: 4, // Runs 4 tests in parallel
});
```

Example 2: Run with Workers via CLI

```
npx playwright test --workers=3
```

Example 3: Enable Fully Parallel Mode in a Test File

```
import { test } from '@playwright/test';

test.describe.configure({ mode: 'parallel' });

test('Test 1', async ({ page }) => {
  await page.goto('https://example.com');
});
test('Test 2', async ({ page }) => {
```

```
    await page.goto('https://example.com/about');  
  });
```

Example 4: Parallel Execution per Project (Multi-Browser)

```
export default defineConfig({  
  projects: [  
    { name: 'Chromium', use: { browserName: 'chromium' } },  
    { name: 'Firefox', use: { browserName: 'firefox' } },  
    { name: 'WebKit', use: { browserName: 'webkit' } },  
  ],  
});
```

Execution Time Comparison:

100 tests x 10 seconds each = 1000 seconds (~16 minutes) With 5 workers = ~200 seconds (~3 minutes) Parallel execution drastically reduces test execution time and improves CI/CD efficiency.

Best Practices for Parallel Execution:

- Keep tests independent.
- Avoid shared global state.
- Use separate test data per test.
- Use fixtures for setup and teardown.